

AK VisionFlow

PHASE 0.2 — UNIVERSAL PRODUCT ARCHITECTURE

Detailed Implementation Specification & Developer Handover

Objective: Transform the current Optical/Medical-coupled Product model into a genuinely universal Product/Service architecture while preserving all existing working Optical and Medical functionality.

Phase boundary: Architectural refactoring only. Owner Mobile remains deferred; Accounting, AI, Marketplace and other later phases remain outside this scope.

1. Phase Objective

- Establish the universal Product domain for Retail, Wholesale, Distribution, Service, Manufacturing, Restaurant/Food, Optical, Eye Clinic/Healthcare and Medical Store/Pharmacy.
- Separate universal product identity from industry-specific attributes and workflows.
- Provide safe migration and compatibility so existing Optical/Medical operations continue working.

2. Phase 0.1 Inputs

- The current Product model contains hard-coded GENERAL, MEDICINE, FRAME and LENS classifications plus Optical/Medical fields such as frameBrand, frameModel, frameColor, frameSize, lensType, lensMaterial, lensCoating, batchNumber and expiryDate.
- Phase 0.1 identified Product Architecture as a primary refactoring target, alongside fixed vertical enums and inconsistent polymorphic patterns.
- These findings are inputs; unrelated findings must not silently expand this phase.

3. Scope

- **In scope:** Product/Service model, classification, categories/brands/units, identifiers, variants, industry extensions, validation, APIs, database migration, compatibility, UI architecture and tests.
- **Out of scope:** Owner Mobile integration, full accounting redesign, new ERP modules, AI redesign, marketplace, white-label implementation and deletion of existing Optical/Medical capabilities.

4. Target Universal Product Architecture

- **Universal Product:** tenant/company ownership, name, description, product/service classification, SKU/internal code, active status, category, brand, unit, identifiers, pricing/tax references, inventory configuration and audit metadata.
- **Service:** first-class sellable item that does not require stock movements.
- **Variant:** optional child structure for size, color, material or other differentiators.
- **Industry Extension:** explicit tenant-safe mechanism for vertical attributes without permanent Optical/Medical columns in the universal Product entity.

5. Industry Extension / Attribute Engine

- Industry-specific fields belong to an industry module/pack rather than Universal Core.
- Attribute metadata should define key, label, data type, required/optional state, validation, visibility and owning industry/module.
- Use structured relational storage where reporting, indexing or integrity requires it; avoid an uncontrolled arbitrary attribute store.
- Optical attributes must not become mandatory for Retail; medicine batch/expiry must not become mandatory for Service.

6. Product Classification

- Replace hard-coded industry meaning in ProductType with a universal classification architecture.
- Support broad concepts such as physical product and service, with future bundle/kit support.
- Industry packs may define Frame, Lens or Medicine as domain classifications while Universal Core remains industry-neutral.
- Every legacy classification must have an explicit target mapping.

7. Legacy Compatibility

- Do not perform a destructive rewrite.

- Keep existing Optical/Medical API contracts where required and use adapters where old fields map to new structures.
- Every legacy field must be classified as Universal Core, Industry Extension, Derived/Deprecated or Owner Decision.
- Maintain rollback capability until migration and regression verification are complete.

8. Database Requirements

- Create a before/after entity map before schema implementation.
- Identify every foreign key, query, validation and report depending on ProductType or Optical/Medical fields.
- Prefer additive/transitional migrations; do not drop legacy columns until all consumers are migrated and approved.
- Preserve tenant isolation and apply the common branch-scope convention to branch-sensitive Product operations.
- Make migrations safe against accidental repeated execution.

9. Backend/API Requirements

- Introduce a universal Product domain/service boundary.
- Centralize product business rules in the service/domain layer rather than controllers.
- Create/read/update operations support universal fields plus controlled industry extensions.
- Validation is industry/configuration-aware, not permanently Optical/Medical-specific.
- Regression-test existing Optical/Medical endpoints and payload expectations.

10. Frontend Requirements

- Universal Product forms show universal fields by default.
- Industry fields appear only when the relevant industry/module is enabled.
- Optical/Medical users retain required operational fields and workflows.
- Do not weaken existing role/route controls.
- UI consumes the universal domain/API rather than duplicating industry business logic.

11. Inventory & Transaction Compatibility

- Physical products continue supporting inventory and stock movements.
- Services do not create meaningless stock movements.
- Medicine batch/expiry and Optical frame/lens information remain available through extensions.
- Sales and Purchase flows must resolve the same product identity before and after migration.
- Do not silently absorb unrelated warehouse/stock-transfer branch-scope work into this phase.

12. Security & Multi-Tenancy

- All Product and extension reads/writes remain tenant-scoped.
- Branch-sensitive operations enforce the established branch-access convention.
- Industry extension records inherit the parent Product authorization boundary.
- Generic attribute endpoints cannot permit cross-tenant access.
- Test unauthorized access explicitly rather than relying on UI restrictions.

13. Testing Strategy

- **Unit:** product/service creation, classification, attributes, validation and mapping.
- **Integration:** API → service → database.

- **Migration:** representative legacy Optical/Medical records before/after mapping.
- **Regression:** products, sales, purchases, inventory, optical orders and medicine workflows.
- **Security:** cross-tenant and relevant cross-branch access.
- **Negative:** duplicate identifiers, invalid attributes, incompatible classifications and unauthorized access.

14. Required Deliverables

- Product Architecture Decision Record (ADR).
- Current vs Target Product Entity Map.
- Product/Service domain model.
- Industry Attribute/Extension specification.
- Classification and legacy mapping specification.
- API compatibility plan.
- Database migration and rollback plan.
- Legacy Optical/Medical field mapping matrix.
- Frontend Product UI architecture.
- Test/regression/security matrix.
- Implementation/change log.
- Phase 0.3 handover package.

15. Mandatory Legacy Field Mapping

- Every current Optical/Medical field must be classified before migration.
- Example direction: frameBrand/frameModel/frameColor/frameSize → Optical extension; lensType/lensMaterial/lensCoating → Optical/Lens extension; batchNumber/expiryDate → Medicine/regulated-inventory extension.
- Do not move a field merely because it appears reusable; ownership must reflect domain responsibility and future industry neutrality.

16. Implementation Sequence

- 1) Freeze Phase 0.1 baseline. 2) Map every Product consumer. 3) Document and approve target architecture. 4) Implement new structures without immediately deleting legacy structures. 5) Implement compatibility layer. 6) Test migration in isolated environment. 7) Update backend consumers. 8) Update frontend consumers. 9) Run full regression/security/migration tests. 10) Obtain Product Owner approval before deprecating legacy structures.

17. Guardrails

- Do not turn Phase 0.2 into general code cleanup.
- Do not rewrite Accounting.
- Do not activate Owner Mobile.
- Do not remove or degrade Optical/Medical functionality.
- Do not perform destructive schema changes without an approved migration and rollback plan.
- Do not treat UI presence or passing tests alone as proof of architectural correctness.

18. Acceptance Criteria

- A generic business can create/sell a physical product without Optical/Medical fields.
- A generic business can create/sell a service without inventory assumptions.

- Optical retains frame/lens workflows through extensions.
- Medical/Pharmacy retains medicine workflows through extensions.
- Existing records remain accurate after migration.
- Existing Product/Sales/Purchase/Inventory/Optical/Medical regression tests pass.
- Tenant isolation remains intact and branch-sensitive operations follow the common convention.
- Architecture, migration, compatibility and rollback are documented and approved.

19. Phase Gate

- Completion requires architecture approval, field mapping approval, migration/rollback evidence, API/UI regression evidence and security evidence.
- Compilation or passing legacy tests alone is not sufficient.
- After Product Owner approval, the Universal Product Architecture becomes the foundation for subsequent Universal Core implementation.

Appendix — Developer Checklist

Area	Requirement	Status
Baseline	Phase 0.1 closure confirmed	■
Consumers	All Product consumers mapped	■
Architecture	Target model documented and approved	■
Extensions	Industry ownership documented	■
Classification	Legacy types mapped	■
Database	Before/after schema map approved	■
Migration	Migration + rollback tested	■
API	Universal service boundary verified	■
Frontend	Universal + industry-aware UI verified	■
Inventory	Product/service stock behavior verified	■
Security	Tenant/branch authorization verified	■
Regression	Optical/Medical workflows verified	■
Documentation	ADR, mappings and API docs complete	■
Owner Gate	Product Owner approval obtained	■

Definition of Done: The Product domain is industry-neutral at its core; industry-specific behavior is isolated behind explicit extensions; legacy Optical/Medical functionality is preserved; migration and rollback are proven safely; APIs/UI remain compatible; security/regression evidence is documented; and Product Owner approval is obtained.